UNIVERSIDAD PRIVADA FRANZ TAMAYO FACULTAD DE INGENIERÍA
CARRERA DE INGENIERÍA DE SISTEMAS



"EVALUACION"

Grupo: Kernel-Hitos

Integrantes: Apaza Plata Leonardo Daniel
Centellas Portugal Jose Luis Omar
Tola Huanca Eva Jhenny
Cuaquira Vargas Boris Cristian

Docente: Ing. Soria Soria Ruben Gonzalo

Semestre: Cuarto

Materia: PROGRAMACION III

La Paz-Bolivia
2026

1. IDENTIFICACIÓN DEL PROYECTO

Nombre del Sistema: Sistema de Registro y Control de Solicitudes de Soporte Técnico
Objetivo General: Desarrollar una aplicación web funcional con PHP y Laravel para gestionar solicitudes de soporte técnico, integrando bases de datos MySQL, transferencia de información hacia una base de datos de la red local, contenedorización con Docker, control de versiones con Git y colaboración mediante GitHub.
Descripción del Sistema: Es una plataforma web desarrollada bajo el patrón MVC con el framework Laravel para la gestión centralizada de solicitudes de soporte técnico. Permite registrar incidentes, visualizar la bandeja de entradas de soporte, actualizar dinámicamente sus estados (Pendiente, En proceso, Resuelta) y cuenta con un mecanismo de sincronización que transfiere de manera segura estos registros locales hacia una base de datos alojada en una red local externa. Todo el entorno está preparado para operar en contenedores Docker y orquestado mediante Docker Compose.

2. INTEGRANTES, ROLES Y RESPONSABILIDADES

Nombre Completo	Rol Asignado	Usuario de GitHub
Leonardo apaza	Backend Laravel	Ipzeleonardodanielapazapl-max
Luis centellas	Funcionalidad web	centellasjoseluis2-lab
Jhenny Tola	Bases de datos e integración	jhennytola31
Cristian cuaquira vargas	Docker, despliegue y documentación	Ipzeboriscristiancuaquirava-dot

Detalle de Responsabilidades:

- **Backend Laravel (Integrante 1):** Modelo, migraciones, rutas, controladores y lógica de registro de solicitudes.
- **Funcionalidad Web (Integrante 2):** Formulario de registro, listado de solicitudes y actualización de estados.
- **Bases de Datos e Integración (Integrante 3):** Conexiones MySQL, configuración de la base de datos de destino y envío de registros.
- **Docker, Despliegue y Documentación (Cris):** Dockerfile, Docker Compose, construcción de la imagen, publicación en Docker Hub y documentación técnica.

3. TECNOLOGÍAS UTILIZADAS

- **Lenguaje y Framework Backend:** PHP 8.2 / Laravel
- **Gestor de Base de Datos Local:** MySQL 8.0
- **Contenedorización y Orquestación:** Docker / Docker Compose
- **Control de Versiones y Repositorio:** Git / GitHub
- **Registro de Imágenes Docker:** Docker Hub

4. FUNCIONALIDADES IMPLEMENTADAS

4.1. Registro de solicitudes de soporte técnico

A través de la interfaz web (`welcome.blade.php`), el usuario completa un formulario que captura información esencial: Solicitante, Departamento, Descripción del problema y Nivel de Prioridad. A través del controlador `SolicitudSoporteController.php`, el sistema valida que los datos no estén vacíos y guarda el registro en la base de datos local asignando el estado predeterminado de 'Pendiente'.

4.2. Listado de solicitudes

La aplicación realiza una consulta al modelo `SolicitudSoporte` y recupera todos los registros almacenados en la base de datos MySQL local. Estos datos son renderizados en la vista web, mostrando de manera clara un listado con la información detallada y el estatus actual de cada ticket.

4.3. Actualización del estado de una solicitud

El sistema incluye en su interfaz acciones (botones) vinculadas a rutas y al controlador correspondiente, las cuales permiten que el usuario modifique el ciclo de vida del ticket, realizando la transición de estado a 'En proceso' y, posteriormente, a 'Resuelta' conforme avance la atención técnica.

4.4. Envío y sincronización a la base de datos de la red local

Se cuenta con un módulo de integración que utiliza una segunda conexión de base de datos (`DB_DESTINO`). Este ejecuta una acción que recopila las solicitudes de la BD local y realiza una operación *upsert* (inserción o actualización masiva) en la BD de la red local, actualizando los registros existentes o insertando los nuevos para impedir duplicaciones en la información transferida

5. ARQUITECTURA DEL SISTEMA Y CONTENEDORES

Descripción de Contenedores: La infraestructura definida en el archivo `docker-compose.yml` orquesta dos contenedores clave: un contenedor de base de datos MySQL que provee la persistencia local de la información, y un contenedor de aplicación web, construido a partir del `Dockerfile` alojado en el repositorio, que contiene el entorno de PHP 8.2 y el sistema Laravel en ejecución.
Flujo de Información: Las solicitudes HTTP del cliente son procesadas por el contenedor de Laravel, el cual efectúa operaciones de lectura y escritura CRUD interactuando directamente con el contenedor MySQL local. Cuando se activa el envío de datos, Laravel genera de forma externa una conexión directa mediante la red local hacia la IP de destino del docente para enviar la data consolidada

6. REQUISITOS PREVIOS

- Git (v2.x o superior)
- Docker Engine y Docker Compose
- Composer (opcional para desarrollo local sin Docker)
- PHP >= 8.1 (opcional para desarrollo local sin Docker)

7. VARIABLES DE ENTORNO Y CONFIGURACIÓN (.env)

Descripción de Variables Clave:

```
DB_CONNECTION=mysql
DB_HOST=db
DB_PORT=3306
DB_DATABASE=laravel_local
DB_USERNAME=root
DB_PASSWORD=root

DB_DESTINO_HOST=[IP_DEL_DOCENTE]
DB_DESTINO_PORT=3306
DB_DESTINO_DATABASE=[NOMBRE_BD_DESTINO]
DB_DESTINO_USERNAME=cuaquira_boris
DB_DESTINO_PASSWORD=K_Cuaq9@Nv26
```

8. PREPARACIÓN E INICIO DE BASE DE DATOS LOCAL (MYSQL)

Para iniciar el motor de base de datos, se debe abrir una terminal en el directorio raíz del proyecto y ejecutar el comando `docker-compose up -d`. Esto descargará e inicializará el contenedor de MySQL basado en la configuración del archivo `docker-compose.yml`. La persistencia de los datos está asegurada mediante el mapeo de volúmenes Docker definidos en este mismo archivo, lo que evita la pérdida de las tablas y registros al apagar o reiniciar los contenedores.

9. PREPARACIÓN DEL PROYECTO LARAVEL

Una vez clonado el repositorio, se deben ejecutar los siguientes pasos:

1. Copiar el archivo .env.example y renombrarlo como .env para la configuración local.
2. Ejecutar composer install para descargar las dependencias listadas en el composer.json.
3. Generar la clave de aplicación única de Laravel con el comando php artisan key:generate.
4. Ejecutar las migraciones, incluyendo el esquema 2026_10_17_124538_create_solicitud_soportes_table.php, usando el comando

php artisan migrate para crear la estructura de datos local.

10. DESPLIEGUE CON DOCKER

Comando de construcción de la imagen:
`docker build -t [USUARIO_DOCKERHUB]/laravel-service:hito3 .`
Comando para iniciar los servicios con Docker Compose:
`docker-compose up -d`
Comando para detener los servicios:
`docker-compose down`

11. CONEXIÓN Y ENVÍO DE DATOS A LA BD DE LA RED LOCAL

Parámetros de conexión: La conexión externa se define en el `.env` apuntando al host o IP_DEL_DOCENTE en el puerto 3306, especificando el nombre de la BD destino y usando las credenciales predefinidas (`DB_DESTINO_USERNAME=cuaquira_boris` y `DB_DESTINO_PASSWORD=K_Cuaq9@Nv26`)
Procedimiento de Sincronización (Upsert): Al accionar la transferencia de información, el sistema emplea sentencias *upsert* (como el método `upsert()` o `updateOrCreate` de Laravel). El proceso comprueba los registros según su identificador único (ID local). Si el ID de la solicitud ya existe en el servidor de destino, sus datos y estado se actualizan; en caso contrario, se inserta un nuevo registro.
Método de Comprobación: Se verifica el éxito de la transferencia mediante un gestor de base de datos SQL conectándose a las credenciales remotas, asegurando que el total de registros y los últimos estados (ej. 'Resuelta') en la base de la red local coinciden plenamente con los listados en la interfaz de la aplicación web local.

12. PUBLICACIÓN EN DOCKER HUB

```
version: '3.8'

services:
  app:
    build:
      context: .
      dockerfile: Dockerfile
    image: centellasjoseluis2/laravel-service:hito3
    container_name: laravel_app
    restart: always
    working_dir: /var/www
    volumes:
      - .:/var/www
    depends_on:
      - db

  db:
    image: mysql:8.0
```

```
    container_name: mysql_db
    restart: always
    environment:
      MYSQL_DATABASE: laravel_local
      MYSQL_ROOT_PASSWORD: root
    ports:
      - "3306:3306"
    volumes:
      - dbdata:/var/lib/mysql

volumes:
  dbdata:
```

Nombre obligatorio de la imagen: [USUARIO_DOCKERHUB]/laravel-service:hito3
Comando para publicar la imagen:
`docker push [USUARIO_DOCKERHUB]/laravel-service:hito3`
Comando de descarga pública:
`docker pull [USUARIO_DOCKERHUB]/laravel-service:hito3`

13. TRABAJO COLABORATIVO Y HISTORIAL DE COMMITS

El control de versiones del proyecto se ha manejado a través de Git y GitHub, respaldado por archivos estructurales como `.gitignore` y `.gitattributes`. El equipo se dividió los flujos de creación del backend (modelos, migraciones, controladores), diseño e implementación de las vistas de listados, la integración base de datos y la orquestación en contenedores. El historial de commits en GitHub evidencia la integración modular, en donde los aportes individuales de validación de datos, despliegue de las conexiones *upsert* y configuración de Docker se unificaron para la entrega final.

14. PRUEBAS Y GUÍA DE VERIFICACIÓN

- Abrir la URL principal del contenedor de Laravel y visualizar el formulario de soporte en la pantalla inicial.
- Registrar un nuevo ticket completando todos los campos; verificar en el listado generado que el mismo figure por defecto en estado "Pendiente".
- Utilizar las acciones de modificación para transitar el ticket al estado "En proceso" y posteriormente pasarlo a "Resuelta".
- Accionar el comando o botón para el envío de información a la red local y constatar mediante un log visual de éxito.
- Conectarse al host de destino MySQL y verificar que la información enviada coincide exactamente, sin presentar entradas duplicadas del mismo ticket.

15. SOLUCIÓN DE PROBLEMAS (TROUBLESHOOTING)

Falla en ejecución de contenedores: Validar que los puertos locales de la máquina anfitriona (ej. el puerto 3306 de la PC) no estén ocupados por una instancia existente de MySQL. Ejecutar docker logs para inspeccionar el punto de error específico.
Error de conexión a Base de Datos local: Comprobar que en el archivo .env los valores como DB_HOST, DB_USERNAME y DB_PASSWORD coincidan estrictamente con las variables definidas en la configuración del servicio de base de datos dentro del archivo docker-compose.yml.
Error al enviar datos a la BD Destino: Si surge un "Connection Refused" o "Timeout", verificar primero haciendo ping a la [IP_DEL_DOCENTE]. Confirmar que la red WiFi/LAN está correctamente vinculada a la red donde se encuentra hospedada dicha IP, y validar que el usuario de destino (cuaquira_boris) posea permisos de acceso remoto vigentes.
https://github.com/centellasjoseluis2-lab/PRO_413_EVAL_HITO3_KERNELHITOS/blob/main/docker-compose.yml
https://github.com/centellasjoseluis2-lab/PRO_413_EVAL_HITO3_KERNELHITOS/blob/main/docker-compose.yml
https://github.com/centellasjoseluis2-lab/PRO_413_EVAL_HITO3_KERNELHITOS/blob/main/docker-compose.yml